When LRU Delivers Nothing: An Anatomy of KV-Cache Tiering Policies for Multi-Turn LLM Serving

Lin Liqian, Beijing University of Posts and Telecommunications

2026-06-15

Contents

Abstract

Multi-turn LLM serving turns the KV cache from a transient tensor into persistent session state: prefixes evicted between turns must either be recomputed or recovered from a lower memory tier. This report measures how much value vLLM's native GPU-to-CPU/CXL-like offloading path can deliver, and why existing policy interfaces fail to reliably capture it. Using controlled single-node experiments, we first show that local CPU recovery reduces second-turn TTFT from 661 ms to 158 ms, while a gentle NUMA proxy for CXL-like memory adds no measurable penalty in the tested unsaturated regime. Under constrained tier capacity, however, stock LRU and ARC behave equivalently and deliver zero complete prefix recoveries on our multi-turn workload. A simple chain-aware prototype demonstrates that preserving complete prefix chains can recover useful session state and reduce store traffic, but subsequent adversarial experiments expose starvation, survivorship bias, and stub-equilibrium failure modes. We conclude that future KV-tiering systems should treat KV cache as session-structured state, exposing request/session context, prefix completeness, and optional application hints to policy layers.

1 Introduction

Large language model serving is increasingly becoming a state-management problem. In a single-turn request, the KV cache can be treated as a temporary execution artifact: it is allocated during decoding, consumed by the attention kernel, and released after the request finishes. In multi-turn serving, however, this view becomes incomplete. A long conversation prefix may be needed again after an idle interval. If its KV cache has been discarded, the system must recompute the prefix before serving the next turn; if it has been preserved in a second memory tier, the system may recover the session state by loading it back. In this sense, multi-turn serving turns the KV cache from a transient tensor into cross-turn session state.

This shift makes KV-cache tiering an important but subtle systems problem. Modern inference engines already provide the basic mechanism to move KV blocks between GPU memory and a lower tier such as CPU memory. However, the existence of an offloading path does not by itself answer the central policy question: under limited second-tier capacity, which KV blocks should be preserved? Classical cache policies such as LRU and ARC are block-centric. They rank individual cache blocks by recency or access history. Multi-turn prefix recovery has a different value structure. A small number of isolated blocks is often not useful; what matters is whether the system can preserve a sufficiently complete prefix chain so that the next turn avoids recomputing the long prefix.

This report studies that gap in the context of vLLM's native KV offloading path. We focus

on a single-node GPU-to-CPU/CXL-like second tier, rather than a full disaggregated cluster. By CPU/CXL-like second tier, we mean a lower memory tier that is slower and larger than GPU memory; in our experiments, we use cross-socket NUMA memory as a controlled proxy for a CXL-attached tier, following the methodology of Pond (Li et al. 2023). Section 2 characterizes the proxy' s fidelity and its limits. This scope is intentional. It lets us isolate a narrow but important question: when KV offloading is available, can the policy layer actually convert second-tier capacity into useful multi-turn prefix recovery?

Our first observation is that the second tier is valuable. In our prefix-recovery experiments (§ 3), recomputing a long prefix places the second-turn TTFT in the recomputation regime, while recovering the prefix from the offloaded tier moves it into a much lower latency regime. Second-turn TTFT drops from 661 ms under recomputation to 158 ms under local CPU recovery —a 4.2× reduction. Under our NUMA-based remote-memory proxy, the remote tier does not introduce a measurable additional penalty in this workload. This does not prove that all CXL-like tiers are free; it shows that a second tier can hold substantial value when the transfer path is not the bottleneck.

The second observation is that this value is not automatically captured by classical cache policies. Even under favorable multi-turn workloads, the offloading path may issue far more store operations than load operations —21:1 in our most reuse-friendly setting. More importantly, when second-tier capacity becomes constrained, LRU and ARC can fail to preserve complete reusable prefixes. In our policy experiments (§ 4), LRU and ARC exhibit equivalent behavior on the tested workload family and deliver no complete prefix recovery, while a simple chain-aware policy can recover a small number of complete prefixes and reduce store traffic measured in transfer operations. This result should not be read as a final policy victory. The more important point is that prefix recovery is not a block-level cache-hit problem; it is a chain-completeness problem.

The third observation is that naive chain awareness is still not enough. Through a sequence of policy prototypes and failed experiments (§ 5), we identify several structured failure modes. A policy that protects early-arriving chains can succeed when early arrivals happen to be revisited, but fails when early arrivals are one-shot sessions. A policy that protects too aggressively can starve the store path and reject almost all new writes. A policy that spreads capacity evenly may produce many small prefix stubs but no complete recoveries. These failures suggest that the main design axis is not simply LRU versus a new replacement rule. The deeper tension is between concentrating limited capacity on a few complete, likely-revisited prefix chains and spreading capacity thinly across many incomplete chains.

This report therefore takes a measurement-and-mechanism perspective rather than presenting a production-ready replacement policy. We ask three research questions. First, how much value can a second memory tier provide for multi-turn prefix recovery, and when does remote recovery remain cheap enough to preserve that value? Second, can classical block-centric policies such as LRU and ARC exploit this value under constrained second-tier capacity? Third, if they fail, what mechanisms explain the failure, and what design principles should guide future KV-tiering policies?

We make four contributions.

1. We quantify the value of a second memory tier for multi-turn prefix recovery. In our controlled workload, local CPU recovery reduces second-turn TTFT by 4.2×

compared with recomputation, and our NUMA proxy shows no measurable remote-memory penalty under the tested, non-saturated transfer regime.

2. We show that classical block-centric policies can fail to convert second-tier capacity into useful prefix recovery. On our constrained multi-turn workload family, LRU and ARC behave equivalently and deliver no complete prefix recovery, despite the availability of the offloading data path.
3. We provide a failure taxonomy for prefix-chain tiering policies. Our prototypes expose starvation under over-protection, two forms of survivorship bias, and a stub equilibrium in which many sessions retain small prefix fragments but no session receives a useful complete recovery.
4. We identify interface and design implications for future KV-tiering systems. The policy layer needs session identity, prefix-chain completeness, optional application hints, and hardware-aware cost models. Our results suggest that future systems should treat KV cache as persistent session state, rather than as a collection of independent cache blocks. We have proposed the minimal interface change upstream (vllm-project/vllm#45405).

Relation to prior work. Cross-node KV-cache disaggregation systems such as Mooncake (Qin et al. 2025) move prefix KV across machines over RDMA at cluster scale; we study the complementary single-node question of what the policy layer can deliver from a local second tier once an offloading path already exists. Tang et al. (Tang et al. 2024) integrate a real ASIC-CXL device with a GPU in a single server and show that the CXL–GPU interconnect matches CPU–GPU transfer latency and bandwidth, establishing the hardware feasibility of CXL KV-cache storage and a 30% batch-size gain under a fixed TTFT SLO. Their save path stores a request's KV only at completion and does not address eviction under constrained capacity; we take the hardware feasibility as given and study the orthogonal policy question —across idle intervals between turns, how limited second-tier capacity should be admitted, protected, and evicted. Our per-turn cost framing (§ 3) is consistent with their production ROI model, which likewise decomposes prefill cost by a new-prompt ratio and a load time, but we operate at the policy layer rather than the hardware layer. Our NUMA-proxy methodology follows Pond (Li et al. 2023), while our conclusions are scoped to a gentle, unsaturated remote-memory regime. TraCT (Yoon et al. 2025) pushes the CXL direction to rack-scale disaggregated serving, using CXL shared memory as both a KV-transfer substrate and a rack-wide prefix-aware KV cache to remove the RDMA/NIC hop between prefill and decode workers. In contrast, this report isolates the local policy and interface layer above a single-node CPU/CXL-like tier: how to turn limited lower-tier capacity into useful multi-turn prefix recovery, and what request/session semantics the runtime must expose for future policies to do so robustly.

2 Background and Testbed

2.1 vLLM's native KV offloading path

We study vLLM's v1 offloading architecture, which separates the tiering machinery into three layers. A connector intercepts scheduler and worker events: when GPU prefix-cache blocks are evicted, their contents become candidates for storage in the second

tier; when a new request's prefix lookup misses in GPU memory, offloaded blocks may be loaded back. An offloading manager owns the second-tier block pool and its bookkeeping. Inside the manager, a pluggable cache policy decides which offloaded blocks to evict when the tier is full. Policies are selected by name through the connector's extra-configuration dictionary, and the stock implementations are LRU and ARC. Transfers execute on dedicated CUDA streams, allowing loads to overlap with ongoing computation.

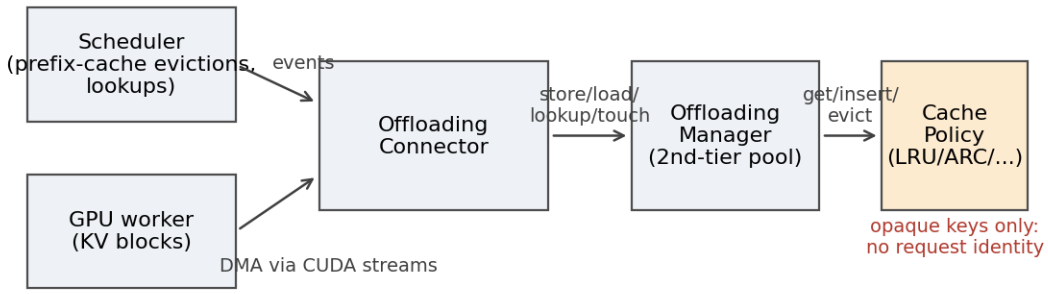

Figure 1: The three-layer offloading path. The cache policy sees opaque keys only; request identity (added in recent main) stops at the manager.

Two properties of this design matter for our study. First, store admission is atomic: to admit a batch of blocks, the policy must evict enough blocks in one call; if it cannot, the entire batch is silently dropped. Second, stores are deduplicated against the tier's current contents, so repeated GPU evictions of an unchanged prefix generate little new store traffic.

2.2 An information-starved policy interface

The policy slot is deliberately narrow: a policy sees opaque block keys and five operations (get, insert, remove, touch, evict). The request context that accompanies manager calls carries no request identity, and the manager drops even that context before invoking the policy. A prefix-chain-aware policy must therefore reconstruct chain structure heuristically from key arrival patterns. We note that as of vLLM's main branch (checked June 2026, after our experiments on v0.21.0), the request context has gained a request identifier and the store path has gained a request-level offload mode —but the manager still drops the context before invoking the cache policy, so the policy slot remains information-starved. We have proposed forwarding this context upstream (vllm-project/vllm#45405); this report provides the measurements motivating that proposal. For our prototypes we added two minimal, backward-compatible forwarding hooks (guarded by attribute checks) that pass the request context to policies that opt in; we verified end-to-end that application-supplied hints injected through request-level transfer parameters reach the policy layer through this path. We return to the interface question in § 8.4.

2.3 Hardware testbeds

Policy experiments (§ 4, § 5) run on a laptop-class RTX 4060 (8 GB) under WSL2, using Qwen2.5-3B-Instruct-AWQ with eager execution and 16-token KV blocks. The GPU block

pool and the second-tier capacity are deliberately constrained (400 GPU blocks; 0.15–0.3 GiB tier) so that both the GPU prefix cache and the second tier experience realistic eviction pressure at small scale.

Tier-value experiments (§ 3) run on a dual-socket server with an RTX 4090D. Following the methodology of Pond (Li et al. 2023), we use cross-socket NUMA memory as a controlled proxy for a CXL-attached tier: the offloaded pool is first-touch allocated on the remote node by a CPU-pinned process, and we verify residency throughout each run with per-node page counters (the pinned pool is immune to automatic NUMA balancing). We characterize the proxy as gentle: the measured remote latency penalty is at least 1.16× (a lower bound, as page-migration contamination biases it downward), and single-core copy bandwidth is core-limited rather than link-limited at 13.3 GB/s. Conclusions about remote tiers therefore hold for an unsaturated, low-penalty regime; bandwidth-constrained regimes are out of scope (§ 9).

2.4 Workloads and instruments

Our workload generator creates sessions with long synthetic prefixes and one or more follow-up turns separated by idle intervals; the revisit set (which sessions return) and turn count are controlled per experiment. Follow-up prompts reuse the first-turn prompt verbatim as a prefix; we deliberately do not append generated text, since detokenization round-trips can break token-exact prefix matching.

We report three instruments. Per-request streaming measurements give TTFT and inter-token gaps. The engine's lookup path logs, for every request, how many offloaded tokens were recovered and how many GPU-cached tokens preceded them; we adopt this hit-tokens count as the authoritative recovery metric. Transfer activity is counted in operations (store and load jobs). A methodological note: an earlier version of our tooling parsed transfer descriptors as per-block entries; cross-checking against the engine's hit-tokens log revealed that the entries are scatter-gather segments, not blocks. All counts in this report use the corrected, operation-level accounting. We mention this because the correction changed our interpretation of intermediate results, and because we believe instrument validation against engine-internal ground truth deserves to be a reported practice rather than a private one.

3 The Value of a Second Tier

3.1 Setup

Six sessions, each with a ~9,216-token prefix, arrive staggered by 10 s on the 4090D testbed; each session is revisited once after a 30 s idle interval. The GPU block pool holds roughly three prefixes, so a session's prefix is reliably evicted from GPU memory between its turns. We compare four arms, three runs each: **A** (no offloading; second turn recomputes the prefix), **S** (offloading to local CPU memory), **R** (offloading to remote-socket memory, CPU also remote —the CXL proxy), and **P** (pool resident on the remote socket, computation pinned back to the local socket —isolating memory placement from CPU placement).

3.2 Results

Arm	t2 TTFT median (ms)	per-run medians	range
A (recompute)	661	659 / 661 / 667	[574, 681]
S (local CPU)	158	157 / 158 / 168	[139, 184]
R (remote, CPU remote)	154	154 / 154 / 168	[141, 183]
P (remote pool, CPU local)	165	168 / 165 / 161	[156, 201]

Table 1: Second-turn TTFT across four arms (A/S/R/P), three runs each.

Recomputation places second-turn TTFT at a median of 661 ms; local CPU recovery places it at 158 ms —a 4.2× reduction with zero overlap between arms. The remote arms are statistically indistinguishable from local recovery (R: 154 ms, P: 165 ms; within-arm spread ±10 ms). A debug-instrumented run confirms the mechanism: exactly one tier-to-GPU load per second turn, so the fast TTFTs are tier recoveries, not residual GPU cache hits.

Two further observations qualify and sharpen this result. First, the absence of a remote penalty is consistent with transfer/computation overlap on per-transfer CUDA streams and an unsaturated cross-socket link; it demonstrates that a low-penalty second tier preserves the full recovery benefit, not that all CXL-class tiers are free. Second, even in this maximally reuse-friendly setting —every session revisits —the offloading path issued 21× more store operations than load operations, foreshadowing the selectivity problem that § 4 and § 5 examine under capacity constraints.

3.3 Where recomputation waste lives

A negative structural result motivates the multi-turn focus. In vLLM's v1 scheduler, newly arriving requests that do not fit simply queue; preemption is triggered only when a running request grows and allocation fails. For long-context requests, decode-phase growth is small relative to the prompt, so admission arithmetic leaves enough headroom that growth-triggered preemption essentially never fires: arrival pressure converts entirely into queueing delay, not recomputation. "Long context × scheduler preemption" is, in this engine, a structurally empty combination. The recomputation cost of long contexts instead lives between turns —in prefix-cache eviction during idle intervals —which is precisely the cost that § 3.2 shows a second tier can remove.

On slower hardware the calculus differs: in earlier experiments on the 4060 testbed with short contexts and high decode growth, preemption-triggered recomputation chunks inflated co-batched requests' P99 inter-token latency by 4×, an interference effect that offloading eliminated; on the 4090D the same effect vanishes because the prefill/decode cost ratio collapses. We use this contrast only to motivate the hardware-aware framing of § 8.6.

4 Classical Policies Deliver Zero

4.1 Setup and a chain-aware strawman

We now constrain the tier. On the laptop testbed, six sessions with ~2,048-token prefixes revisit cyclically (a symmetric workload: every session returns), while the tier holds

roughly two complete prefix chains. Under this configuration the tier experiences genuine eviction pressure —and the policy slot, for the first time, actually decides outcomes. We compare LRU, ARC, and a chain-aware prototype, **chain-v1**, built on three rules: blocks of one prefix chain are evicted together (whole-chain eviction, tail-first); eviction order among chains is most-recently-stored first (the anti-cyclic direction); and the two oldest chains are guarded —if satisfying an eviction request would break the guard, the store is rejected outright.

4.2 Results

Policy	t2 TTFT median (s) ×3	load ops ×3	store ops ×3	complete recoveries ×3
LRU	0.864 / 0.871 / 0.792	0 / 0 / 0	70 / 70 / 70	0 / 0 / 0
ARC	0.892 / 0.837 / 0.838	0 / 0 / 0	70 / 70 / 70	0 / 0 / 0
chain-v1	0.602 / 0.556 / 0.566	3 / 3 / 4	52 / 52 / 52	3 / 3 / 3

Table 2: LRU / ARC / chain-v1 on the symmetric cyclic-revisit workload, three runs each. LRU and ARC are bit-identical; chain-v1 is the only policy with non-zero complete recoveries.

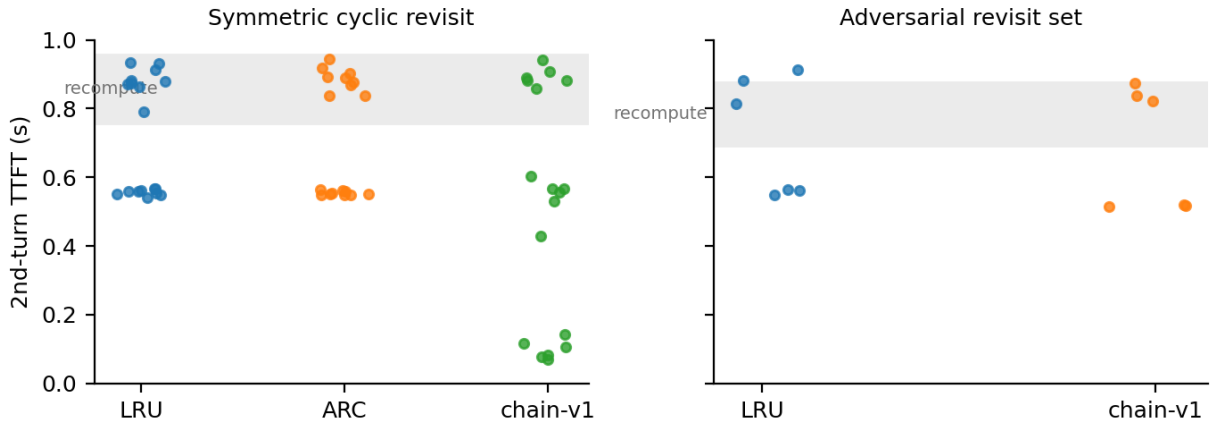

Figure 2: Per-session second-turn TTFT. Left: symmetric cyclic revisit —chain-v1's recovered sessions drop to the 0.07–0.14 s band while LRU/ARC stay in the recompute band. Right: adversarial revisit set —all policies collapse to the recompute band.

LRU and ARC deliver nothing. Across three runs each, neither policy serves a single load operation; every revisit pays a substantial recomputation. Their transfer traces are not merely similar but bit-identical —70 store operations in every run, identical sequences — meaning ARC's adaptive machinery never produced a single decision that differed from LRU on this workload. Their second-turn TTFT medians (0.79–0.89 s) sit at the recomputation regime.

chain-v1 delivers. In every run it serves 3–4 load operations and produces three complete prefix recoveries in every run —the engine's hit-tokens log shows 2,016–2,256 offloaded tokens recovered with zero GPU-cache contribution —yielding second-turn TTFTs of 0.07–0.14 s for the recovered sessions and a median improvement of ~31%. It does so while issuing ~25% fewer store operations than LRU: the write-rejection rule converts protection into store-traffic selectivity at no cost to hits. On this workload, both metrics improve simultaneously.

4.3 Honest mechanism: early-arrival pinning

The mechanism behind chain-v1's success is less sophisticated than its rules suggest. Its store rejections partially freeze the tier around the earliest-arriving chains; on a symmetric cyclic workload, the earliest arrivals are revisited by construction, so the frozen contents are exactly the right contents. We name this mechanism early-arrival pinning and emphasize that it is a property of the workload-policy pair, not of the policy alone. When we break the alignment —constructing revisit sets in which the earliest sessions are one-shot —chain-v1 collapses to LRU-equivalent zero delivery. That collapse, and what it takes to escape it, is the subject of § 5.

5 Anatomy of Failure

Classical policies fail on this workload, but a naive chain-aware policy is not the answer either. This section dissects three structured failure modes that we encountered while iterating on the chain-aware prototype: store-path starvation, two rounds of survivorship bias, and a stable stub equilibrium. Each prototype iteration was pre-registered with falsifiable predictions before the corresponding experiment, and each failure was traced to a mechanism before the next iteration was designed. We report the failures in this form because the mechanisms, not the prototypes, are the transferable result.

Throughout this section, the workload is the adversarial multi-turn configuration of § 4: ten sessions, of which the six latest-arriving sessions are revisited for three turns while the four earliest are one-shot, under a 0.15 GiB second tier that can hold roughly two complete prefix chains. The authoritative metric is per-request hit-tokens (§ 2): the number of offloaded prefix tokens actually recovered for a revisited request, as reported by the engine's lookup path. We extend chain-v1 (§ 4) in three steps: **v2** adds reuse counting and an application-hint channel; **v3** adds a block-budgeted guard and ghost-chain credit; **v3.1** adds per-request store-batch delimitation.

5.1 The information-theoretic deadlock

The first failure is not a policy bug but an information bound. In a two-turn workload with no external hints, a one-shot session and a to-be-revisited session are observationally identical until the revisit actually arrives. Any online policy must therefore allocate protection before the only distinguishing event has happened; over the space of revisit-set choices, no ranking rule can beat chance.

Our experiments illustrate both sides of this bound. When the randomly drawn revisit set happened to contain the oldest sessions, chain-v1's age-based protection appeared

to succeed —three complete recoveries —but the test had no discriminative power, since age and revisit status were accidentally aligned. We therefore report results only on adversarially constructed revisit sets in which the oldest sessions are one-shot. On those sets, chain-v1 collapses to LRU-equivalent behavior: protection is spent on one-shot sessions, and its write-rejection mechanism turns against the very sessions it was meant to serve.

Design implication. Revisit prediction needs either longer observable history (more than two turns) or semantic hints from the application layer; in their absence, comparing eviction policies on two-turn traces measures luck, not policy quality (§ 8.3).

5.2 Starvation by over-protection

The second failure mode appeared when we combined two individually reasonable mechanisms. v2 preserved chain identity across touches, consolidating each request' s blocks into a single chain; it also retained v1' s protection rule, which guards a fixed number of chains and rejects any store that would require breaking the guard. Under consolidation, the live-chain count early in a run is small —on the order of three —so guarding two chains, while the incoming store batch itself is protected, leaves the evictable set empty. Every eviction request fails; every failed eviction atomically rejects an entire store batch.

The result is a frozen tier. In our runs, v2 logged 1,364 rejected store batches and not a single successful eviction; only 7 store operations completed over the whole run, versus 127 under LRU on the identical workload, and the tier served zero load operations. The deadlock is self-sustaining: rejection prevents new chains from forming, which keeps the chain count low, which keeps the guard fraction total.

Retrospectively, this failure also reinterprets v1' s earlier success. v1' s touch handling fragmented chains, keeping the chain count high and the guard fraction low —the "defect" we fixed in v2 was a load-bearing accident. v1' s wins on favorable workloads came from early-arrival pinning: the tier froze around the first arrivals, which on those workloads happened to be the revisited sessions.

Design implication. Protection must be budgeted in capacity units (blocks), not in policy-level objects (chains), and the eviction path must guarantee a non-empty evictable region; otherwise protection composes with atomic store admission into livelock (§ 8.2, § 8.4).

5.3 Survivorship bias, twice

The third failure mode concerns how a policy learns which chains will be revisited. Our reuse signal incremented a chain' s credit when its blocks were touched by a lookup with no intervening store. This signal is conditioned on survival: a chain that was evicted before its revisit is touched as a miss —there is nothing left to touch —and accumulates no credit. The chains that most need protection are exactly the chains the signal cannot see. We addressed this first round of survivorship bias with ghost chains: chain metadata that outlives the blocks, records missed lookups, and transfers credit when the chain' s blocks are re-stored. This is, in effect, a rediscovery of ARC' s ghost lists at chain rather than block granularity —we note the irony that ARC itself, operating on blocks, was behaviorally indistinguishable from LRU on this workload (§ 4).

Ghost credit, however, exposed a second round of the same bias. Chains that survive as small head stubs are touched on every revisit wave and farm credit continuously; in our logs, long-lived stubs reached reuse counts of 12–16. Rebuilt chains —re-stored after a miss, carrying one or two ghost-inherited credits —rank below the stubs and become the first eviction victims. The credit mechanism, designed to identify valuable chains, systematically protects fragments and sacrifices reconstructions.

Design implication. Reuse credit must be normalized by what the credit was earned on: a touch on a 10% stub is not evidence of recoverable value. Completeness-weighted credit, or admission control that prevents stubs from competing with reconstructions, is required (§ 8.1).

5.4 The stub equilibrium

The final state of the v3/v3.1 prototypes is a stable basin we call the stub equilibrium, and it is jointly maintained by three forces. First, capacity is oversubscribed roughly 3:1 —six revisited sessions of ~140 blocks each contend for a 279-block tier —so at most two complete chains can physically coexist. Second, MRU-direction whole-chain eviction, the mechanism that defeats cyclic scans, kills every reconstruction attempt: a chain re-stored after a miss is by definition the newest object in the tier and therefore the first victim, while old head stubs are systematically spared. Third, credit farming (§ 5.3) makes the ranking agree with the eviction order: stubs outrank reconstructions.

The observable signature is striking in its uniformity. Across both the history-based and the hint-based variants, every revisited session recovers 192–224 hit-tokens per turn — under 10% of the prefix —with zero variation in pattern across runs; no session ever receives a complete recovery. The ceiling experiment shows that this is purely a policy failure: with an unconstrained 4 GiB tier under plain LRU, every revisited session obtains a complete recovery of 2,240+ tokens with zero GPU-cache contribution, at both the second and third turns (12 of 12). Capacity arithmetic bounds what any policy could achieve at 0.15 GiB —about two complete recoveries per wave —and the prototypes deliver none of them. The hit-tokens table also reveals a further degradation across turns: at the third turn, the constrained prototypes recover zero offloaded tokens —even the stubs touched at the second turn have been churned away before the third arrives.

Config	turn	hit-tokens per revisited session (sorted)	complete (⩾2000)
W0 (LRU)	t2	(no CPU hits)	0
W3h (history)	t2	192, 192, 208, 208, 208, 224	0
W3o (hint)	t2	192, 192, 208, 208, 208, 224	0
W5h (v3.1 history)	t2	192, 192, 208, 208, 208, 224	0
W5o (v3.1 hint)	t2	192, 192, 208, 208, 224, 224	0

Config	turn	hit-tokens per revisited session (sorted)	complete (⩾2000)
W4u (4 GiB ceiling)	t2	2240, 2240, 2256, 2256, 2256, 2272	6
W4u (4 GiB ceiling)	t3	2256, 2256, 2256, 2272, 2272, 2272	6

Table 3: Per-session hit-tokens across W-series configurations. Constrained prototypes plateau at 192–224 tokens (the stub equilibrium); the unconstrained 4 GiB ceiling (W4u) recovers all 12 of 12.

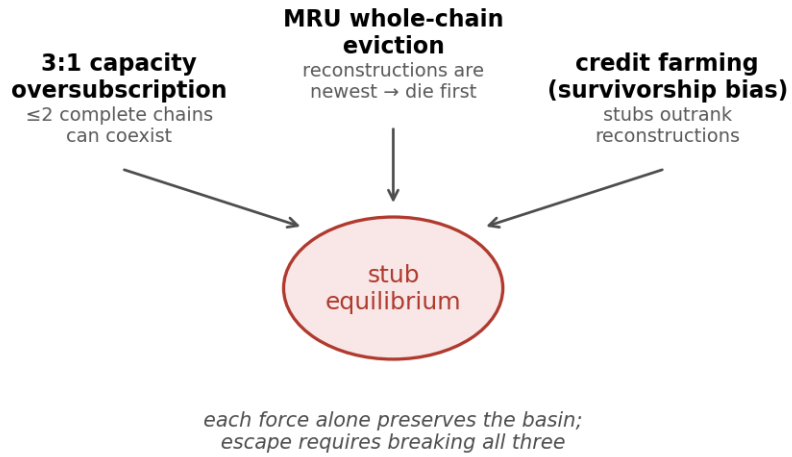

Figure 3: Three forces jointly maintain the stub equilibrium; breaking any one alone leaves the basin intact.

A methodological note: the ceiling run's TTFT distribution alone would have suggested that only half of the revisits recovered fully; the hit-tokens metric shows all of them did, with residual TTFT variance attributable to factors other than recovery completeness (most plausibly load/queueing overlap, which we did not isolate). This is a second demonstration of why § 2.4 treats engine-internal recovery accounting, not end-to-end latency, as the authoritative metric.

Design implication. Escaping the stub equilibrium requires breaking all three forces at once: concentration within a capacity budget, completeness-weighted (not survival-weighted) credit, and an eviction direction that does not structurally select against reconstruction. Fixing any single force leaves the basin intact —which is precisely why three successive prototypes, each fixing one force, all converged to the same equilibrium.

6 The Ceiling

Two control experiments bound what any eviction policy could achieve on the adversarial workload of § 5.

First, capacity arithmetic. At 0.15 GiB the tier holds roughly 279 blocks while a complete chain occupies ~140; no eviction policy, however clairvoyant, can keep more than about two complete chains resident. The per-wave upper bound for complete recoveries is therefore ~2 of 6 revisited sessions —the prototypes' failure is that they deliver zero, not that they deliver fewer than six.

Second, the unconstrained ceiling. With a 4 GiB tier —capacity for every chain in the workload —plain LRU on the identical adversarial workload delivers a complete recovery (2,240+ hit-tokens, zero GPU contribution) for every revisited session at every turn: 12 of 12. Capacity is the only binding constraint we measured; the store path, given the 30 s idle intervals of this workload, accumulates complete chains reliably. We note that end-to-end TTFT in the ceiling run still varied by 7× across these uniformly complete recoveries; we attribute the variance to load and queueing overlap (not isolated in this study), and treat it as further evidence that recovery value must be measured by hit-tokens rather than latency alone.

Whether store-path timing and granularity can limit completeness under shorter idle intervals —where a revisit may arrive before the GPU has gradually evicted, and the tier absorbed, the full chain —is a plausible concern that our workloads do not reach; we flag it as untested in § 8.5 and § 9.

7 When the Second Tier Is Used: A Real-Trace Probe

The experiments so far use synthetic workloads with long, homogeneous prefixes and a deliberately tiny second tier. A natural question is whether the policy regime they expose is an artifact of those choices. To probe this, we replayed real multi-turn conversations from the ShareGPT corpus (Tang et al. 2024 uses the same source) and measured when the second tier is actually exercised.

Real prefixes are short and heavy-tailed. Across 39,583 usable multi-turn sessions (human-first, strictly alternating, at least one follow-up), the first-turn prefix has a median of only 30 tokens and a 90th percentile of 408; the contiguous context a follow-up must recover has a median of 312 tokens and a 99th percentile of 1,607. Only about 15% of sessions carry a prefix long enough (256–4,096 tokens) to be worth replaying at all. The value of prefix recovery is thus concentrated in a long-tail minority of long-context sessions; for the median conversation, recomputation is nearly free.

On a single GPU, the GPU prefix cache absorbs the reuse, and the second tier stays idle. Replaying 12 and then 30 sessions under an LRU second tier, we observed a consistent pattern: the store path is busy (199 and 548 store operations respectively, as the GPU cache evicts prefixes into the tier), but the tier serves zero loads. The engine's lookup checks the GPU prefix cache first and hits there: with short real prefixes and idle intervals (20–25 s) shorter than the GPU cache's eviction horizon, a revisit finds its prefix still resident on the GPU, and never consults the tier. Shrinking the GPU pool to force eviction collides with a hard floor —serving the longest real prefix at a 4,096-token context already requires more blocks than the point at which the cache stops covering the working set. On 8 GB there is no configuration that both serves the real workload and forces the tier into use.

This sharpens, rather than weakens, the report's scope. The policy results of § 4– § 6

apply to the regime in which the second tier is actually on the critical path —i.e., where the working set exceeds GPU-cache capacity. Real ShareGPT traffic on a single small GPU sits outside that regime: the tier is redundant, and which eviction policy it runs is immaterial because it is idle. Entering the regime requires either high concurrency (production batch sizes, where GPU memory is genuinely overwhelmed —the setting in which Tang et al. (Tang et al. 2024) report their 30% throughput gain) or long contexts. Our synthetic workload (a 2,048-token prefix against a 0.15 GiB tier) is precisely an instrument for pushing a single GPU across that threshold so that policy behavior becomes observable. In other words, the synthetic design is not a convenience that sacrifices realism; it is the minimal apparatus that reproduces, on one laptop GPU, a regime that otherwise appears only at production scale.

8 Implications & Future Work

Our experiments suggest that KV-cache tiering for multi-turn LLM serving should not be framed as a simple replacement-policy problem. The basic offloading mechanism is necessary, but it is not sufficient. The main question is how the system should translate limited second-tier capacity into useful prefix recovery. From the measurements and failures in this report, we draw five design implications.

8.1 Preserve complete prefixes, not isolated blocks

The first implication is that prefix completeness should be a first-class policy objective. Classical cache policies optimize at the level of individual blocks. This is natural for many cache systems, but it mismatches the value structure of LLM prefix recovery. A small prefix stub may count as a cache hit in a low-level metric, but it may not significantly reduce second-turn TTFT. Conversely, preserving fewer but more complete prefix chains can be much more valuable than spreading the same capacity across many sessions.

This distinction explains why partial-hit metrics can be misleading. A policy may appear to keep many sessions partially warm, yet fail to deliver any full prefix recovery. In our failure analysis, this appears as a stub equilibrium: each revisited session obtains a small number of offloaded tokens, but none receives enough contiguous prefix state to avoid substantial recomputation. Under constrained capacity, the policy objective should therefore be closer to maximizing useful recovered prefix length per revisited session, not merely maximizing the number of touched or retained blocks. Section 5 uses hit-tokens as the operational form of this metric: it measures how many offloaded prefix tokens are actually recovered for a revisited session, and exposes the difference between complete recoveries and shallow stubs.

8.2 The first design axis is concentration versus uniform poverty

The second implication is that the primary design axis is concentration versus spreading. When limited capacity is spread so thinly that many sessions retain fragments but none receives a useful recovery, we call the resulting regime uniform poverty. With enough second-tier capacity, a broad caching policy may be acceptable. Under tight capacity, however, trying to keep a little bit of every session can be worse than keeping a few ses-

sions well. Multi-turn prefix recovery has a threshold-like character: below some completeness level, the recovered prefix may not change the user-visible latency regime.

This does not mean the system should always protect the oldest chains or the most recent chains. Our chain-aware prototype shows that concentrated protection can work when the protected sessions are later revisited, but the same mechanism fails when protected sessions are one-shot. The lesson is more general: the policy must concentrate capacity, but it must concentrate on the right chains. Concentration without revisit awareness degenerates into early-arrival pinning; uniform spreading degenerates into the stub equilibrium analyzed in § 5.

8.3 Revisit prediction should not rely only on online learning

The third implication is that revisit prediction is necessary, but pure online learning may be the wrong default interface. In a two-turn workload with no external hint, a policy cannot reliably distinguish a one-shot session from a session that will return before the return actually happens. This is an information problem, not merely an algorithmic weakness. A more complex online learner may still lack the signal required to make the right decision early enough.

Online learning also faces survivorship bias. Chains that are evicted before reuse provide little positive feedback, while small surviving stubs may be repeatedly touched and accumulate misleading credit. This can make the system protect incomplete stubs rather than reconstruct complete useful chains. In such settings, adding more learning machinery may not solve the problem unless the observations themselves become more informative.

For this reason, application-level hints are a practical and attractive path. Many serving applications already know whether a request belongs to an active chat session, a one-shot summarization job, an agent workflow, or a likely follow-up interaction. Passing this session-level intent into the KV-tiering policy is a simple heuristic, but it may be more stable and deployable than trying to infer all future reuse from low-level block touches. The goal is not to build an oracle. The goal is to expose cheap semantic signals that the runtime already lacks.

8.4 The policy interface should expose session and chain semantics

The fourth implication concerns system interfaces. A KV offloading backend can provide the data path, but the policy layer also needs the right control-plane information. If the policy only sees anonymous blocks, it is forced to reconstruct session identity and chain structure indirectly. This makes prefix-aware tiering brittle and implementation-dependent. Notably, vLLM's main branch has recently added a request identifier to the offloading context and a request-level offload mode at the store path —evidence that request-level reasoning is the direction the engine is already moving —yet this context still does not cross the manager–policy boundary. Our critique therefore targets that final boundary specifically; we have proposed the corresponding minimal change upstream (vllm-project/vllm#45405).

A better interface should expose at least three pieces of information. First, the policy should know which blocks belong to the same request or session. Second, it should know how blocks form a prefix chain and how complete that chain is. Third, it should be able

to receive optional application hints about expected reuse or session class. These signals would allow the policy to distinguish complete useful prefixes from isolated stubs, and long-lived sessions from one-shot requests.

This does not require the engine to commit to a single policy. Instead, it requires the engine to preserve enough context so that different policies can be implemented cleanly. The policy slot should not only decide which block to evict; it should be able to reason about the unit of value in multi-turn serving.

8.5 Storage-side behavior remains an open question

The fifth implication is more tentative than the others. Eviction policy is one side of the problem; the other side is when and how KV state enters the second tier. In our workloads, with 30-second idle intervals, the store path reliably accumulated complete chains —our ceiling experiment found no storage-side limit. However, the store path saves blocks gradually, as the GPU prefix cache evicts them, and we conjecture that under shorter idle intervals or higher GPU-cache pressure, a revisit may arrive before a chain has been stored completely. If so, selective saving, proactive demotion, and completeness-aware store admission would become as important as replacement. We flag this as a measurable hypothesis rather than a finding: our data neither confirms nor refutes it, and § 9 lists it as untested. We note that frequency-based store admission has recently appeared in vLLM' s store path, suggesting storage-side selectivity is an active direction upstream.

8.6 Toward a hardware-aware but unified framework

Finally, our measurements suggest that there may not be a single universally optimal KV-tiering policy across all platforms. On slower GPUs, the dominant value of offloading may be eliminating recomputation interference and reducing tail TBT. On faster GPUs, short-context recomputation may become cheap enough that the same effect disappears. In long-context multi-turn serving, the dominant value shifts toward prefix recovery latency. With narrower second-tier bandwidth, we expect store amplification and selective saving to become dominant; however, we have not measured this bandwidth-limited regime in this report. With tighter capacity, prefix completeness and admission control dominate.

However, this does not mean the design space is fragmented. The same framework can organize these cases. A KV-tiering system should ask four questions: What is the cost of recomputation on this hardware? What is the cost of loading from the second tier? Which sessions are likely to return? How complete must a prefix be before recovery becomes useful? Hardware changes the answer to these questions, but not the questions themselves.

This leads to a practical direction for future work: build a session-aware and hint-aware KV-tiering policy that combines simple application hints with completeness-aware protection and store admission. Such a policy should avoid relying on block recency alone, avoid overfitting to early arrivals, and avoid treating small stubs as equivalent to useful prefix recovery. The broader lesson is that multi-turn LLM serving requires the runtime to manage KV cache as persistent session state, not merely as a collection of pages.

9 Limitations

Our conclusions carry the following scope restrictions. All experiments use a single 3B-parameter AWQ-quantized model in eager mode on one engine version; absolute latencies are specific to this configuration, and our claims rest on regime separations (recomputation vs. recovery; complete vs. stub) rather than absolute numbers. The hardware base is one laptop GPU and one dual-socket server; n = 3 per cell, which we consider sufficient only because the separations are categorical and the transfer-operation traces are deterministic across runs. Synthetic workloads carry controlled prefix lengths, revisit sets, and idle intervals; § 7 reports a real-trace probe that situates these synthetic choices, but production traffic at scale —with its own concurrency and revisit structure —remains future work. The remote tier is a NUMA proxy characterized as gentle —latency penalty lower-bounded at 1.16×, link unsaturated —so the zero-penalty result does not extend to bandwidth-constrained CXL configurations, which we have not measured. Store traffic is reported in transfer operations, not bytes. Store-path completeness limits under short idle intervals are conjectured (§ 6) but not measured. Finally, the policy interface we study is one engine' s; our interface critique (§ 8.4) describes a class of information-starvation problems we expect to generalize, but the specific patch points do not.

References

Li, Huaicheng, Daniel S. Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, et al. 2023. "Pond: CXL-Based Memory Pooling Systems for Cloud Platforms." In Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS). Vancouver, BC, Canada. https://doi.org/10.1145/3575693.3578835.

Qin, Ruoyu, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. "Mooncake: Trading More Storage for Less Computation —a KVCache-centric Architecture for Serving LLM Chatbot." In 23rd USENIX Conference on File and Storage Technologies (FAST 25), 155–70. Santa Clara, CA: USENIX Association. https://www.usenix.org/conference/fast25/presentation/qin.

Tang, Yupeng, Runxiang Cheng, Ping Zhou, Tongping Liu, Fei Liu, Wei Tang, Kyoungryun Bae, Jianjun Chen, Wu Xiang, and Rui Shi. 2024. "Exploring CXL-based KV Cache Storage for LLM Serving." In Machine Learning for Systems Workshop at NeurIPS 2024.

Yoon, Dongha, Younghoon Min, Hoshik Kim, Sam H. Noh, and Jongryool Kim. 2025. "TraCT: Disaggregated LLM Serving with CXL Shared Memory KV Cache at Rack-Scale." https://arxiv.org/abs/2512.18194.